

A Scalable Python Pipeline for IMU-Based Behavioral Clustering

Chuqi Wang

June 17, 2026
revised August 24, 2026

Abstract

This report describes a Python reimplementaion of the MATLAB behavioral clustering pipeline. The core representation is preserved: 300 ms IMU windows are encoded as channel-wise histograms, converted to CDFs, and compared via CDF-L1 distance (equivalent to 1D Wasserstein/EMD). The main addition is a set of scalable Affinity Propagation variants—random-sampled, coreset, k-means, stratified, sparse, and two-level—that avoid the $O(N^2)$ memory cost of full AP while staying close to its output. All methods are evaluated on eight datasets using silhouette score and ARI against full AP; full AP is now computed on seven of the eight datasets (raised from three), from a single dependency-version-pinned run. Beyond cluster-level agreement, random-sampled AP is validated against full AP on population-level behavioral summaries—occupancy and transition-matrix correlation—with an honest look at where it does *not* agree: frame-level and single-exemplar identity. Random-sampled AP is currently the best practical default; k-means and stratified sampling are promising alternatives for datasets where rare behaviors matter.

1 Project Goal

This project analyzes mouse behavior from wearable inertial measurement unit (IMU) recordings. The central computational task is to partition continuous motion sensor data into short behavioral windows and assign each window to an unsupervised behavioral cluster. These clusters can then be used for downstream analyses, such as comparing behavioral motif usage across control and Parkinsonian mice or across different arena conditions.

The original MATLAB pipeline performs Affinity Propagation clustering using an EMD-based similarity matrix. However, the MATLAB implementation becomes expensive because it computes many pairwise EMD distances and stores an $N \times N$ similarity matrix, where N is the number of behavioral windows. The Python pipeline keeps the same conceptual distance metric while introducing vectorized preprocessing, CDF-based EMD computation, approximate/exact nearest-neighbor search with FAISS, and several scalable AP variants.

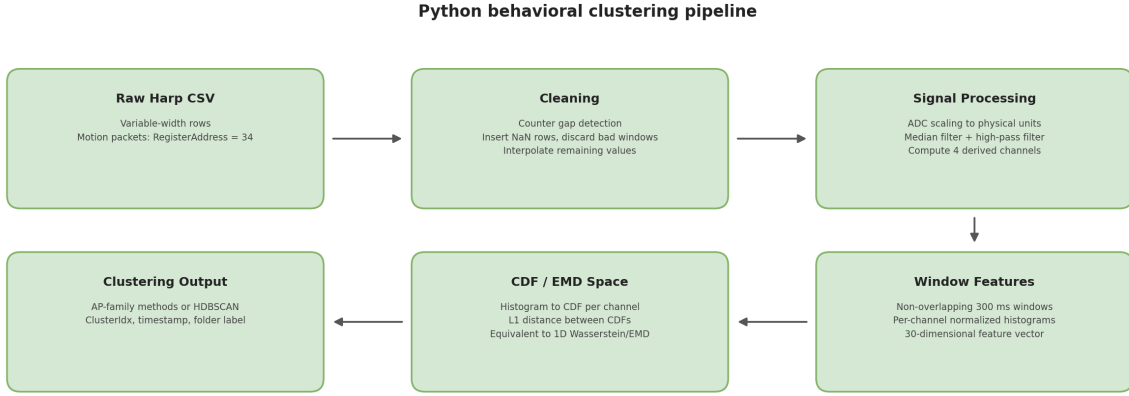


Figure 1: Overview of the Python pipeline from raw Harp CSV files to behavioral cluster labels.

2 Data Overview

2.1 Raw Harp Motion Data

The raw input is a Harp motion CSV file. Rows can have variable width, so `prepare_test_data.py` first pads all rows into a consistent table. Motion packets are identified by:

$$\text{RegisterAddress} = 34. \quad (1)$$

Only these rows contain IMU motion samples. After filtering to motion rows, the Python loader extracts:

$$\text{raw_motion} \in \mathbb{R}^{N_{\text{samples}} \times 10}, \quad (2)$$

with columns

$$[\text{ax}, \text{ay}, \text{az}, \text{gx}, \text{gy}, \text{gz}, \text{mx}, \text{my}, \text{mz}, \text{counter}].$$

The first three columns are accelerometer readings, the next three are gyroscope readings, the next three are magnetometer readings, and the last column is the packet counter. Timestamps and folder/session labels are stored separately. The magnetometer columns (`mx`, `my`, `mz`) are present in the raw data but are not used by the current pipeline.

2.2 Missing-Packet Detection and Cleaning

The packet counter wraps around from 127 to 0. Consecutive motion packets should therefore have counter difference:

$$\Delta c = (c_{t+1} - c_t) \bmod 128 = 1. \quad (3)$$

If $\Delta c > 1$, then $\Delta c - 1$ motion packets were dropped. The preprocessing script inserts placeholder rows with NaN sensor values and linearly interpolated timestamps. This preserves temporal alignment before windowing.

The cleaning rule is window-based. Since one behavioral window contains 60 samples, corresponding to 300 ms at 200 Hz, the script checks missingness inside each 60-row block. If more

than 10% of samples in a window contain NaNs in the sensor columns, the entire window is discarded. Remaining NaNs are linearly interpolated. The cleaned motion data are finally trimmed to a multiple of 60 samples.

3 Signal Processing

After loading the cleaned motion rows, `clustering_pipeline.py` applies signal processing designed to mirror the MATLAB pipeline.

3.1 Scaling to Physical Units

Raw accelerometer and gyroscope ADC values are scaled as:

$$\mathbf{a} = \text{raw_accel} \times \frac{4}{32768}, \quad (4)$$

$$\boldsymbol{\omega} = \text{raw_gyro} \times \frac{1000}{32768}. \quad (5)$$

The accelerometer range is $\pm 4g$, and the gyroscope range is ± 1000 degrees per second.

3.2 Filtering and Derived Signals

The pipeline applies a clipped-boundary median filter with kernel size 7. It then applies a zero-phase first-order Butterworth high-pass filter at 0.5 Hz to separate body acceleration from slower gravitational/postural components. The code constructs the four channels used for histogram features:

$$y_{\text{GA}} = y - y_{\text{BA}}, \quad (6)$$

$$z = \text{median-filtered vertical acceleration}, \quad (7)$$

$$z_{\text{gyro}} = \text{median-filtered gyroscope z-axis}, \quad (8)$$

$$a_{\text{tot}} = \sqrt{x_{\text{BA}}^2 + y_{\text{BA}}^2 + z_{\text{BA}}^2}. \quad (9)$$

The fourth histogram channel uses $\log(a_{\text{tot}})$, with a small numerical floor to avoid $\log(0)$.

4 Windowing and Histogram Representation

4.1 Behavioral Windows

The processed time series is split into non-overlapping windows:

$$W_i = \{60 \text{ consecutive samples}\}. \quad (10)$$

At 200 Hz, 60 samples correspond to:

$$\frac{60}{200} = 0.3 \text{ s} = 300 \text{ ms}. \quad (11)$$

Thus, each row of the clustering input represents one 300 ms behavioral segment.

4.2 MATLAB-Compatible Histogram Bins

The current Python code uses bin edges recovered from the MATLAB `hardHistNew.mat` thresholds. The resulting feature vector has 30 dimensions:

$$11 + 11 + 6 + 2 = 30. \quad (12)$$

Table 1: Histogram channels and bin counts used by the current Python implementation.

Channel	Number of bins	Meaning
y_{GA}	11	Gravity/postural component of the y-axis acceleration
z	11	Median-filtered z-axis acceleration
z_{gyro}	6	Median-filtered z-axis angular velocity
$\log(a_{\text{tot}})$	2	Log total body acceleration magnitude

For a window W_i and channel c , the pipeline counts how many of the 60 samples fall into each bin:

$$n_{i,c,b} = \#\{x_t \in W_i : x_t \text{ falls in bin } b\}. \quad (13)$$

The count histogram is normalized by window length:

$$h_{i,c,b} = \frac{n_{i,c,b}}{60}. \quad (14)$$

Therefore, each per-channel histogram is a discrete probability distribution:

$$\sum_b h_{i,c,b} = 1. \quad (15)$$

The four channel histograms are concatenated:

$$\mathbf{h}_i = [\mathbf{h}_{i,y_{\text{GA}}}, \mathbf{h}_{i,z}, \mathbf{h}_{i,z_{\text{gyro}}}, \mathbf{h}_{i,\log(a_{\text{tot}})}] \in \mathbb{R}^{30}. \quad (16)$$

4.3 Worked Example: Data at Each Processing Stage

Tables 2 and 3 show five consecutive raw samples and their processed equivalents from the `short_comparison_test` dataset (starting at $t \approx 339.47$ s). Figure 2 shows the resulting 30-dimensional histogram feature vector for the 300 ms window centred at $t = 339.63$ s, chosen as a representative example with values spread across multiple bins in each channel.

Table 2: Stage 1 — raw ADC samples. All ten sensor columns are shown. **ax–gz**: accelerometer and gyroscope integer counts; **mx–mz**: magnetometer counts (not used by the pipeline); **ctr**: packet counter (wraps 0–127).

t (s)	ax	ay	az	gx	gy	gz	mx	my	mz	ctr
339.473	−7692	−923	1368	3253	930	−897	−30	−18	−34	27
339.478	−8041	−1740	1876	4787	−1121	1091	−30	−18	−34	28
339.483	−8823	−1796	2243	2636	−2255	1893	−30	−18	−34	30
339.488	−8099	−849	2510	404	−2231	1947	−33	−17	−36	31
339.493	−6685	−350	4486	−265	−1887	3228	−33	−17	−36	32

Table 3: Stage 2 — processed signals after ADC scaling, median filtering, and high-pass filtering. Units: y_{GA} and z in g ; z_{gyro} in deg s^{-1} ; $\log(a_{tot})$ dimensionless.

t (s)	y_{GA}	z	z_{gyro}	$\log(a_{tot})$
339.473	−0.145	0.306	−27.4	−0.776
339.478	−0.143	0.306	33.3	−0.769
339.483	−0.142	0.306	57.8	−0.763
339.488	−0.141	0.306	59.4	−0.756
339.493	−0.140	0.306	98.5	−0.788

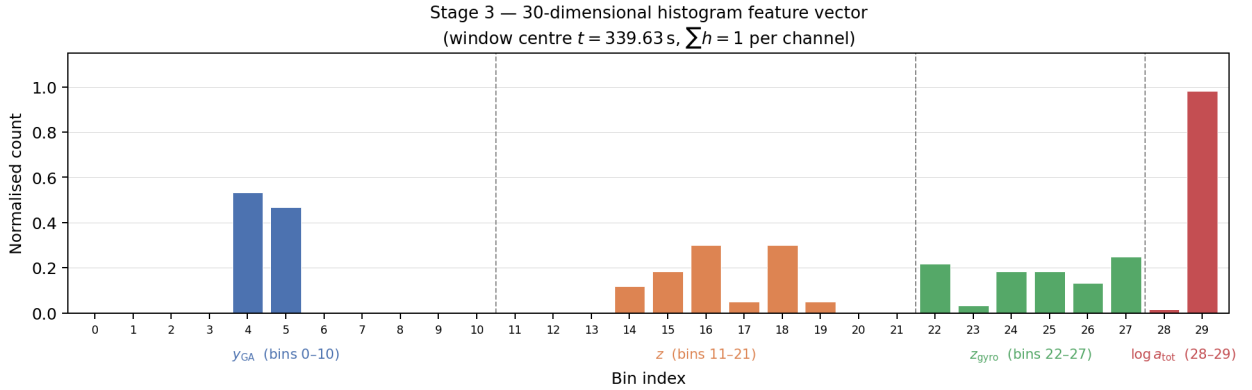


Figure 2: Stage 3 — 30-dimensional normalised histogram feature vector for the 300 ms window centred at $t = 339.63$ s. Each colour corresponds to one sensor channel; dashed lines mark channel boundaries. The y_{GA} channel splits across bins 4–5, indicating the mouse posture oscillated near the boundary between two ranges during this window. The z channel shows a bimodal pattern (bins 16 and 18), and all six z_{gyro} bins are occupied, reflecting varied angular velocity. Each channel sums to 1.

5 CDF Transformation and EMD Distance

5.1 CDF Representation

Earth Mover’s Distance measures how much “probability mass” must be moved to transform one distribution into another [4]. In one dimension, Wasserstein-1 distance between two normalized histograms can be computed efficiently as the L1 distance between their cumulative distribution functions (CDFs).

For each channel c , define the CDF of window i as:

$$H_{i,c,b} = \sum_{r=1}^b h_{i,c,r}. \quad (17)$$

The Python code additionally divides the CDF L1 sum by $(B_c - 1)$, where B_c is the number of bins for channel c . This gives each channel comparable weight even when it has a different number of bins.

5.2 Per-Channel EMD Approximation

For two windows i and j , the per-channel CDF distance is:

$$d_c(i, j) = \frac{1}{B_c - 1} \sum_{b=1}^{B_c} |H_{i,c,b} - H_{j,c,b}|. \quad (18)$$

This is the one-dimensional Wasserstein/EMD distance for uniformly spaced bins, up to the normalization used in the implementation.

5.3 Total Behavioral Distance

The total distance between two behavioral windows is the sum across channels:

$$d(i, j) = \sum_{c=1}^4 d_c(i, j). \quad (19)$$

In code, the four per-channel CDF vectors are concatenated into a single feature vector $\mathbf{C}_i \in \mathbb{R}^{30}$, so the total distance reduces to a single L1 call:

$$d(i, j) = \|\mathbf{C}_i - \mathbf{C}_j\|_1. \quad (20)$$

The per-channel normalization by $(B_c - 1)$ is absorbed into the concatenated representation, so the single L1 call correctly weights all four channels.

5.4 AP Similarity

Affinity Propagation requires similarities rather than distances. The MATLAB pipeline uses negative squared EMD similarity:

$$s(i, j) = -d(i, j)^2. \quad (21)$$

The Python AP methods follow the same similarity convention, but the preference setting depends on the AP variant. For full AP, the preference is set to:

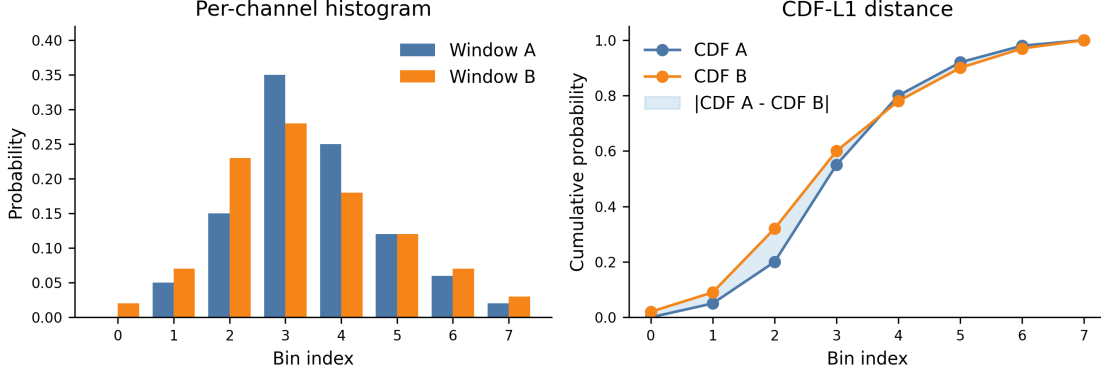


Figure 3: Toy example showing how two window-level histograms are transformed into CDFs. In one dimension, the L1 area between CDFs gives the Wasserstein/EMD distance used by the clustering methods.

$$p = \min_{i,j} s(i,j), \quad (22)$$

matching the current MATLAB setting `param = min(s(:,3))` in `VPAPPAxes.m`, where column 3 of `s` stores the pairwise similarity values. For random-sampled AP, coreset AP, sparse AP, k-means-sampled AP, and stratified AP, the code estimates the same global minimum similarity by sampling 500,000 random pairs from all N windows. Two-level AP is the main exception: it uses median affinity within local blocks and again among candidate exemplars, because a global minimum preference is too conservative at those smaller local scales.

6 Role of FAISS

FAISS [2] is not itself the behavioral clustering algorithm in this pipeline. It is used as a fast nearest-neighbor engine operating on the CDF feature vectors computed in the previous section. In the AP sampled, coreset, k-means sampled, stratified, sparse, and two-level methods, AP finds a smaller set of exemplars. FAISS then answers:

For this window, which exemplar has the smallest CDF-L1 distance?

Mathematically, if $\mathcal{E} = \{\mathbf{e}_1, \dots, \mathbf{e}_K\}$ is the final exemplar set, every window i receives label:

$$\ell_i = \arg \min_{k \in \{1, \dots, K\}} \|\mathbf{C}_i - \mathbf{e}_k\|_1. \quad (23)$$

Thus FAISS accelerates assignment and KNN graph construction; it does not replace AP's exemplar selection.

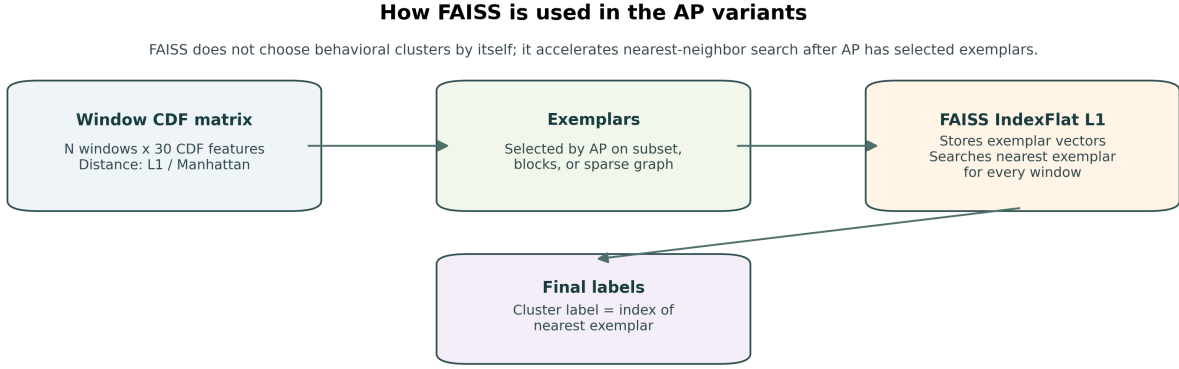


Figure 4: FAISS usage in the AP variants. AP selects exemplars; FAISS stores those exemplar vectors and performs fast nearest-neighbor search under L1 distance to assign every window to its closest exemplar.

7 Why the Python Pipeline Is Faster Than the MATLAB-Style Approach

The speed improvement comes from two separate changes. First, the distance computation is vectorized: each window is represented as a CDF vector, and the EMD-like distance between two windows is computed by a simple L1 distance between CDF vectors. This avoids repeatedly calling a MATLAB/MEX EMD routine inside nested loops.

Second, the scalable AP variants avoid constructing the full dense $N \times N$ similarity matrix for large datasets. Full AP is still available as a reference method, but the scalable methods run AP on a subset, on local blocks, or on a sparse KNN graph, then use FAISS to assign all windows to the final exemplars. The assignment step scales with the number of windows times the number of exemplars rather than all pairs of windows.

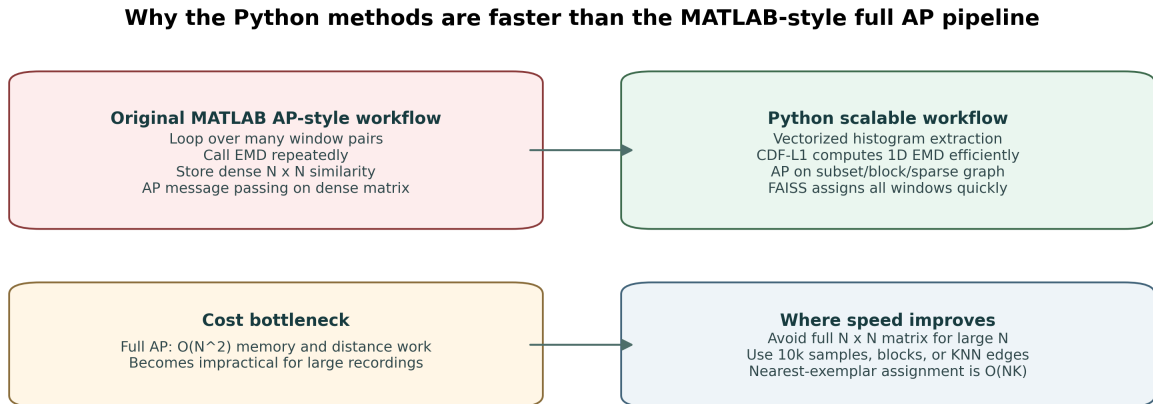


Figure 5: Main computational differences between the MATLAB-style full AP workflow and the scalable Python variants. The largest gain comes from avoiding dense all-pairs AP computations when N is large.

8 Affinity Propagation Algorithm

Affinity Propagation (AP) [1] is an exemplar-based clustering algorithm. Rather than requiring the number of clusters to be specified in advance, AP discovers exemplars — representative data points that best summarize groups of similar points — by iteratively passing messages between all pairs of points until a consistent set of exemplars emerges.

8.1 Problem Formulation

Given N data points with a pairwise similarity matrix $S \in \mathbb{R}^{N \times N}$, AP seeks a subset $\mathcal{E}$ of exemplars and an assignment $e_i \in \mathcal{E}$ for each point i that maximizes the total similarity:

$$\mathcal{E}^* = \arg \max_{\mathcal{E}} \sum_{i=1}^N s(i, e_i), \quad (24)$$

subject to the self-consistency constraint that every exemplar must represent itself:

$$e_k = k \quad \text{for all } k \in \mathcal{E}. \quad (25)$$

In the current pipeline, the similarity between two windows is the negative squared CDF-L1 distance:

$$s(i, k) = -d(i, k)^2 = -\|\mathbf{C}_i - \mathbf{C}_k\|_1^2. \quad (26)$$

The diagonal $s(k, k) = p$ is the *preference* parameter, which controls how many exemplars are found. A higher preference (closer to zero) yields more exemplars and therefore more clusters; a lower preference (more negative) yields fewer.

8.2 Message-Passing Equations

AP maintains two $N \times N$ message matrices that are updated alternately at each iteration.

Responsibility $r(i, k)$. The responsibility $r(i, k)$ is sent from point i to candidate exemplar k . It reflects how much point i “prefers” k as its exemplar relative to all other candidates:

$$r(i, k) \leftarrow s(i, k) - \max_{k' \neq k} \{s(i, k') + s(i, k')\}. \quad (27)$$

A large positive $r(i, k)$ means k is a better candidate for i than any alternative.

Availability $a(i, k)$. The availability $a(i, k)$ is sent from candidate exemplar k to point i . It reflects how “willing” k is to be an exemplar, given the support it receives from other points.

For off-diagonal entries ($i \neq k$):

$$a(i, k) \leftarrow \min \left(0, r(k, k) + \sum_{i' \notin \{i, k\}} \max(0, r(i', k)) \right). \quad (28)$$

The minimum with zero enforces that only already-supported candidates can attract more points. For the self-availability ($i = k$):

$$a(k, k) \leftarrow \sum_{i' \neq k} \max(0, r(i', k)). \quad (29)$$

Damping. To prevent numerical oscillations during message passing, each new message is mixed with its previous value using damping factor $\lambda \in [0.5, 1)$:

$$r \leftarrow (1 - \lambda) r_{\text{new}} + \lambda r_{\text{old}}, \quad (30)$$

$$a \leftarrow (1 - \lambda) a_{\text{new}} + \lambda a_{\text{old}}. \quad (31)$$

The current pipeline uses $\lambda = 0.9$ for all AP variants (both the scikit-learn calls and the custom sparse AP implementation). This is higher than the scikit-learn default of 0.5, and was chosen to improve convergence stability on the high-dimensional CDF feature space.

8.3 Convergence and Assignment

At each iteration, exemplars are identified by checking the diagonal of $R + A$. Point k is an exemplar when:

$$r(k, k) + a(k, k) > 0. \quad (32)$$

The algorithm converges when the set of exemplars remains unchanged for a fixed number of consecutive iterations (default: 15 in scikit-learn) or when the maximum iteration count is reached. After convergence, every point i is assigned to the exemplar that maximises its criterion:

$$e_i = \arg \max_k \{r(i, k) + a(i, k)\}. \quad (33)$$

8.4 Pseudocode

Algorithm 1 summarizes the full AP procedure.

8.5 Time and Memory Complexity

Full AP requires storing and updating two $N \times N$ message matrices, giving:

Resource	Cost
Memory	$O(N^2)$
Time per iteration	$O(N^2)$
Total time	$O(T \cdot N^2)$

For $N = 25,000$ and $T \approx 200$ iterations, full AP requires approximately 18 GB of memory and several minutes of compute. This is the fundamental bottleneck that motivates all scalable AP variants described in Section 9.

8.6 Parameter Settings Used in This Pipeline

Table 4 lists the AP hyperparameters used across all variants in the current implementation.

All variants use $\lambda = 0.9$, which is higher than the scikit-learn default of 0.5. A higher damping value slows convergence but reduces oscillation, which improves stability when running AP on high-dimensional CDF feature vectors.

Table 4: AP hyperparameter settings used in the current pipeline. Two-level AP uses different iteration limits at each level because block-level AP operates on smaller subproblems.

Method	λ	max_iter	conv_iter	Preference
AP Full	0.9	1000	15	exact min of $N \times N$ affinity
AP Sampled	0.9	1000	15	global min [†]
AP Coreset	0.9	1000	15	global min [†]
AP K-Means	0.9	1000	15	global min [†]
AP Stratified	0.9	1000	15	global min [†]
AP Sparse (custom, $K = 1000$)	0.9	1000	15	global min [†]
AP Two-level (L1)	0.9	300	15	per-block median affinity
AP Two-level (L2)	0.9	500	30	median among candidate exemplars

[†] Estimated from 500,000 random pairs drawn from all N windows.

Algorithm 1 Affinity Propagation

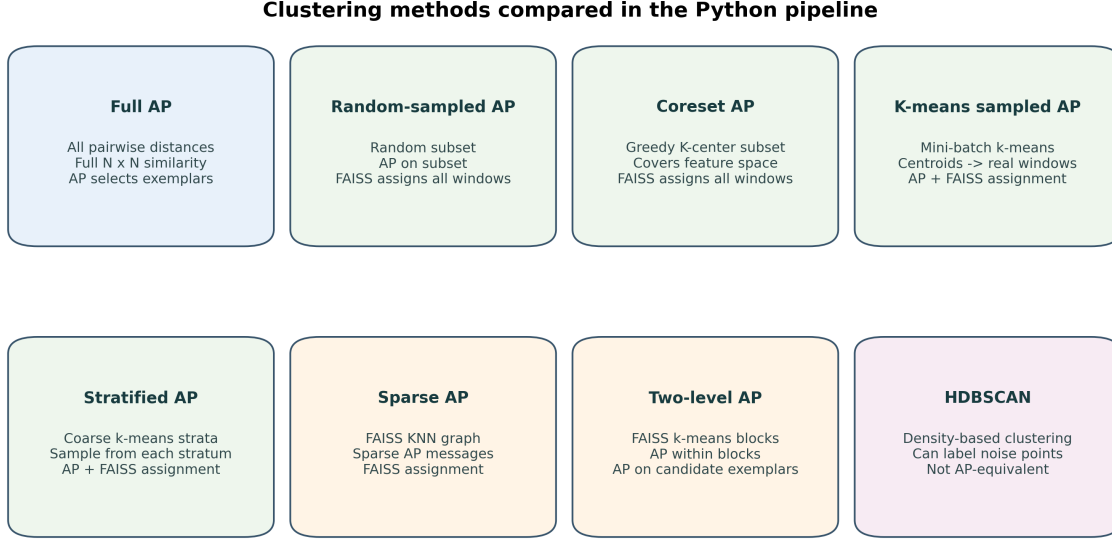
Require: Similarity matrix $S \in \mathbb{R}^{N \times N}$, preference p , damping $\lambda = 0.9$, **max_iter** $T = 1000$, **convergence_iter** $C = 15$

Ensure: Exemplar set $\mathcal{E}$, assignment labels $\{e_i\}$

```
1:  $S[k, k] \leftarrow p$  for all  $k$  ▷ set diagonal preference
2:  $R \leftarrow \mathbf{0}_{N \times N}$ ,  $A \leftarrow \mathbf{0}_{N \times N}$  ▷ initialize messages
3:  $\text{stable} \leftarrow 0$ 
4: for  $t = 1$  to  $T$  do
5:   // Update responsibilities
6:   for each  $i$  do
7:      $\text{AS}[k] \leftarrow A[i, k] + S[i, k]$  for all  $k$ 
8:      $R_{\text{new}}[i, k] \leftarrow S[i, k] - \max_{k' \neq k} \text{AS}[k']$  for all  $k$ 
9:   end for
10:   $R \leftarrow (1 - \lambda) R_{\text{new}} + \lambda R$ 
11:  // Update availabilities
12:  for each  $k$  do
13:     $\text{Rp}[i'] \leftarrow \max(0, R[i', k])$  for all  $i' \neq k$ 
14:     $A_{\text{new}}[i, k] \leftarrow \min(0, R[k, k] + \sum_{i' \notin \{i, k\}} \text{Rp}[i'])$  for all  $i \neq k$ 
15:     $A_{\text{new}}[k, k] \leftarrow \sum_{i' \neq k} \text{Rp}[i']$ 
16:  end for
17:   $A \leftarrow (1 - \lambda) A_{\text{new}} + \lambda A$ 
18:  // Check convergence
19:   $\mathcal{E} \leftarrow \{k : R[k, k] + A[k, k] > 0\}$ 
20:  if  $\mathcal{E}$  unchanged from previous iteration then
21:     $\text{stable} \leftarrow \text{stable} + 1$ 
22:  else
23:     $\text{stable} \leftarrow 0$ 
24:  end if
25:  if  $\text{stable} \geq C$  then break
26:  end if
27: end for
28:  $e_i \leftarrow \arg \max_k \{R[i, k] + A[i, k]\}$  for all  $i$  ▷ final assignment
29: return  $\mathcal{E}$ ,  $\{e_i\}$ 
```

9 Clustering Methods

The current comparison script evaluates seven AP-family methods plus one HDBSCAN reference method. AP methods produce cluster exemplars; HDBSCAN identifies dense regions and may label sparse points as noise.



AP-family methods select exemplars. FAISS is used for fast nearest-exemplar assignment or KNN graph construction; HDBSCAN follows a separate density-based objective.

Figure 6: Visual summary of the clustering methods currently compared in `Python Pipeline`. The AP-family methods differ mainly in how they reduce the cost of exemplar selection before assigning all windows to final exemplars.

9.1 Full Affinity Propagation

Full AP [1] is the closest Python equivalent to the MATLAB algorithm. It computes the full pairwise distance matrix:

$$D \in \mathbb{R}^{N \times N}, \quad (34)$$

converts it to a similarity matrix $S = -D^2$, fills the diagonal with the AP preference, and runs scikit-learn’s [7] `AffinityPropagation` with precomputed affinities. The preference is set to the exact minimum of the $N \times N$ affinity matrix, matching the MATLAB `AcclCluster` setting. It is the most faithful reference method, but requires $O(N^2)$ memory and $O(N^2)$ work per AP iteration, so it is skipped for datasets with $N > 25,000$ in the batch comparison.

9.2 Random-Sampled AP

Random-sampled AP reduces the AP cost by selecting a subset of windows:

1. Randomly sample **10,000 windows**.

2. Run full AP on the sampled windows.
3. Extract AP exemplars from the sample.
4. Use FAISS exact L1 search to assign every original window to its nearest exemplar.

The preference is estimated from 500,000 random pairs drawn from *all* N windows (not just the 10,000 sample), keeping the preference scale consistent with full AP. This is the default scalable AP method in the current pipeline.

9.3 Coreset-Sampled AP

Coreset-sampled AP replaces random sampling with Greedy K-Center selection. The goal is to select representative windows that cover the feature space, including rare behaviors that random sampling may miss.

1. Pick an initial point.
2. Iteratively add the point farthest from the selected set until **10,000 windows** are selected.
3. Run AP on the selected coreset.
4. Assign all windows to the nearest exemplar using FAISS.

The preference uses the same global min estimate as random-sampled AP (500,000 random pairs from all N windows). This method can improve coverage of rare behavioral modes, but it may not always best match full AP.

9.4 Sparse AP

Sparse AP builds a K -nearest-neighbor graph over all N windows (FAISS IVFFlat) and runs AP message-passing on that graph rather than on a dense $N \times N$ matrix. The CLI default is $K = 100$; the batch comparison in Section 12 used $K = 1,000$. Memory drops from $O(N^2)$ to $O(NK)$. The custom message-passing implementation symmetrizes the graph and adds self-loops at the preference value before iterating.

In practice, sparse AP tends to over-fragment: because windows can only communicate with their K nearest neighbours, two behaviorally related windows that happen to sit far apart in feature space may never exchange messages and end up in separate clusters. This is a structural limitation of the approach, not a tuning problem—increasing K helps but requires proportionally more memory.

9.5 Two-Level AP

Two-level AP is designed to let all windows participate without forming a global dense affinity matrix.

1. Use FAISS k-means to partition all N windows into $M = \max(4, \lfloor N/2500 \rfloor)$ blocks of $\sim 2,500$ windows each (FAISS k-means, `niter=20`).
2. Run full AP inside each block (`max_iter=300`, `conv_iter=15`, `preference = per-block median affinity`) to obtain local candidate exemplars.

3. Run a second AP over all candidate exemplars M_{cand} (`max_iter`=500, `conv_iter`=30, preference = median affinity among candidates).
4. Assign all windows to the nearest final exemplar using FAISS exact L1 search.

This method reduces AP cost from roughly $O(N^2)$ to approximately $O(N^2/M + M_{\text{cand}}^2)$ over M blocks, where M_{cand} is the number of candidate exemplars collected from all blocks. The median preference at both levels is an intentional design choice: using the global minimum preference would collapse each local block to a single exemplar, leaving too few candidates for the second-level AP.

9.6 K-Means-Sampled AP

K-means-sampled AP uses mini-batch k-means to select central representative windows:

1. Run mini-batch k-means with $k = 10,000$ to partition all N windows (`n_init`=3, `batch_size`=min(4096, N), `max_iter`=100).
2. Map each centroid to the nearest real window using FAISS exact L1 search.
3. Run AP on the 10,000 selected real windows.
4. Assign all windows to the nearest AP exemplar using FAISS.

The preference uses the same global min estimate as random-sampled AP (500,000 random pairs from all N windows). Compared with random sampling, this method selects cluster-central rather than boundary points, which can improve ARI against full AP.

9.7 Stratified-Sampled AP

Stratified AP first partitions the full dataset into coarse strata and then samples from each stratum.

1. Run mini-batch k-means with $k = 200$ coarse strata (`n_init`=3, `batch_size`=min(4096, N), `max_iter`=100).
2. Draw approximately equal numbers of windows from each stratum until **10,000 windows** are selected in total (~ 50 per stratum).
3. Run AP on the stratified sample.
4. Assign all windows to the nearest exemplar using FAISS.

The preference uses the same global min estimate as random-sampled AP (500,000 random pairs from all N windows). By sampling equally from each stratum, this method ensures that rare behaviors occupying small fractions of the recording are represented in the AP sample, regardless of their frequency.

9.8 HDBSCAN Reference Method

HDBSCAN [3] is included as a density-based reference. It clusters CDF features under L1 distance without needing a preference parameter, and assigns low-density windows a noise label (-1) rather than forcing them into a cluster. Reproducing MATLAB AP results is not the goal—HDBSCAN optimises a different objective—but it serves as a useful sanity check and exploratory tool, particularly for identifying whether the recording contains genuinely distinct dense behavioral modes.

10 Evaluation Metrics

10.1 Number of Clusters and Noise

For AP methods, every window is assigned to a cluster, so the noise percentage is 0%. For HDBSCAN, some windows may be labeled as noise and are excluded from cluster-size statistics.

10.2 Silhouette Score

Silhouette score [5] measures how close a point is to its own cluster compared with other clusters. The code computes silhouette using L1 distance on CDF features, subsampling up to 2,000 valid windows for speed. Note that HDBSCAN silhouette scores are not directly comparable to AP-family scores: HDBSCAN may achieve high silhouette by labeling ambiguous boundary windows as noise and retaining only well-separated points, whereas AP assigns every window to a cluster. The interpretation used in the report is:

Silhouette range	Interpretation
> 0.5	good
$0.25\text{--}0.5$	reasonable
< 0.25	poor

10.3 ARI Against Full AP

Adjusted Rand Index (ARI) [6] is used when full AP is available. Full AP is treated as the MATLAB-like reference. ARI is interpreted as:

ARI value	Meaning
1	identical clustering
0	agreement expected by chance
< 0	worse than chance

11 Datasets

The current batch comparison evaluates eight datasets.

Table 5: Datasets included in the current method comparison. Duration is computed as $N \times 300$ ms.

Dataset	N	Duration	Description
<code>short_comparison_test</code>	1,195	~6 min	Short subset of <code>comparison_test</code> for quick testing
<code>comparison_test</code>	13,799	~69 min	Full MATLAB-equivalent reference recording
<code>mp_mouse_1_jul</code>	19,714	~99 min	MitoPark Parkinson’s mouse, July session
<code>control_mouse_1_jul</code>	23,902	~120 min	Control mouse, July session (age-matched)
<code>control_mouse_1_oct</code>	23,906	~120 min	Control mouse, October session (age-matched)
<code>mp_mouse_1_oct</code>	23,866	~119 min	MitoPark Parkinson’s mouse, October session
<code>still_test</code>	23,877	~119 min	Low-motion baseline recording
<code>moving_test</code>	47,028	~235 min	High-motion activity recording

12 Results

12.1 Cluster Counts and Silhouette Scores

Tables 6 and 7 summarize the current cached comparison results, from a single version-pinned run (`requirements.txt`; numpy 2.5.2, scikit-learn 1.9.0, faiss-cpu 1.15.0) performed on 2026-08-24. Cluster counts and silhouette scores are shown separately for readability. Full AP is skipped when the dataset has more than 25,000 windows (raised from an earlier 20,000-window threshold after moving to a 48 GB machine; see the reproducibility note at the end of this section). The accompanying figures use abbreviated axis labels from the plotting code (e.g., *AP twolevel*, *AP kmeans*, *AP stratified*), which correspond to the full method names used in the tables and text.

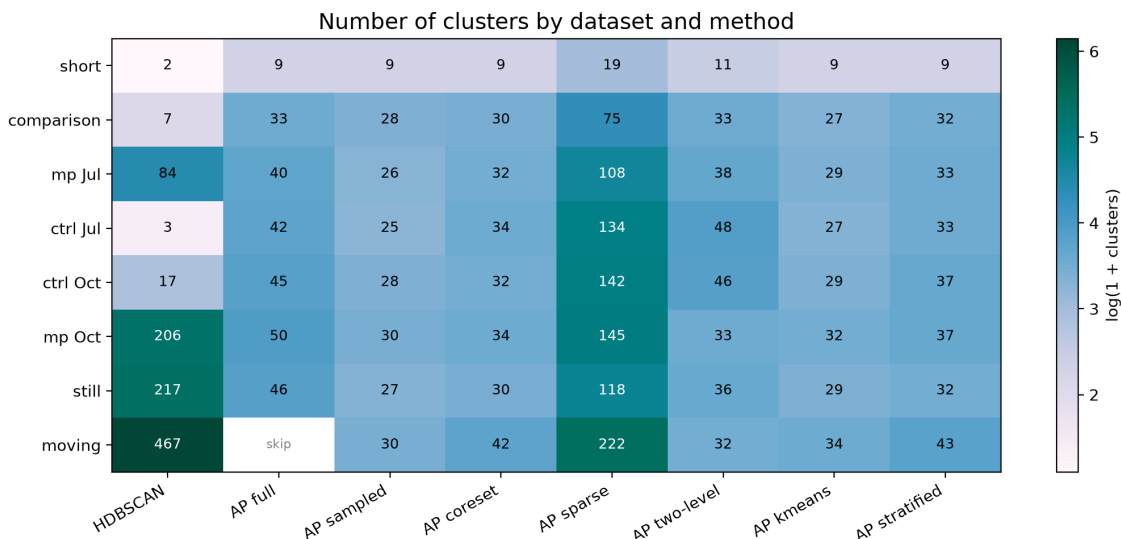


Figure 7: Heatmap of the number of clusters found by each method. Sparse AP and HDBSCAN tend to produce many more clusters than the AP sampled family.

Table 6: Number of clusters found by each method. A dash indicates that AP Full was skipped because the dataset exceeded the full-AP size threshold. High cluster counts in Sparse AP and HDBSCAN reflect known limitations of those methods rather than desirable results.

Dataset	HDBSCAN	AP Full	AP Sampled	AP Coreset	AP Sparse	AP Two-level	AP K-Means	AP Stratified
short_comparison_test	2	9	9	9	19	11	9	9
comparison_test	7	33	28	30	75	33	27	32
mp_mouse_1_jul	84	40	26	32	108	38	29	33
control_mouse_1_jul	3	42	25	34	134	48	27	33
control_mouse_1_oct	17	45	28	32	142	46	29	37
mp_mouse_1_oct	206	50	30	34	145	33	32	37
still_test	217	46	27	30	118	36	29	32
moving_test	467	–	30	42	222	32	34	43

Table 7: Silhouette score by method. Bold indicates the highest value per row. A dash indicates that AP Full was skipped. HDBSCAN scores are not directly comparable with AP scores; see Section 10.2.

Dataset	HDBSCAN	AP Full	AP Sampled	AP Coreset	AP Sparse	AP Two-level	AP K-Means	AP Stratified
short_comparison_test	0.302	0.152	0.152	0.152	0.133	0.160	0.152	0.152
comparison_test	0.277	0.148	0.136	0.129	0.117	0.149	0.143	0.144
mp_mouse_1_jul	0.057	0.196	0.194	0.225	0.153	0.197	0.227	0.199
control_mouse_1_jul	0.408	0.132	0.136	0.135	0.085	0.128	0.136	0.144
control_mouse_1_oct	0.108	0.122	0.145	0.155	0.059	0.130	0.139	0.134
mp_mouse_1_oct	0.353	0.213	0.217	0.207	0.135	0.209	0.231	0.232
still_test	0.300	0.208	0.190	0.186	0.179	0.168	0.194	0.190
moving_test	0.574	–	0.279	0.313	0.203	0.235	0.264	0.239

12.2 Agreement with Full AP

Full AP is now available for seven of the eight datasets (up from three in the previous version of this report), after raising the full-AP size threshold from 20,000 to 25,000 windows – see the reproducibility note at the end of this section. Only `moving_test` ($N = 47,028$) still exceeds the threshold. Table 8 reports ARI against full AP. The strongest agreement is usually achieved by sampled AP variants, especially random, k-means, and stratified sampling. The same column abbreviations are used as above.

With full AP now computed on four additional datasets, the picture from the original three-dataset comparison holds: no single sampled variant dominates, but random, k-means, and stratified sampling consistently land in the 0.33–0.49 ARI range while HDBSCAN and sparse AP trail behind (HDBSCAN in particular collapses to near-zero agreement on `control_mouse_1_jul` and `comparison_test`, where it finds only 3–7 clusters against full AP’s 33–42).

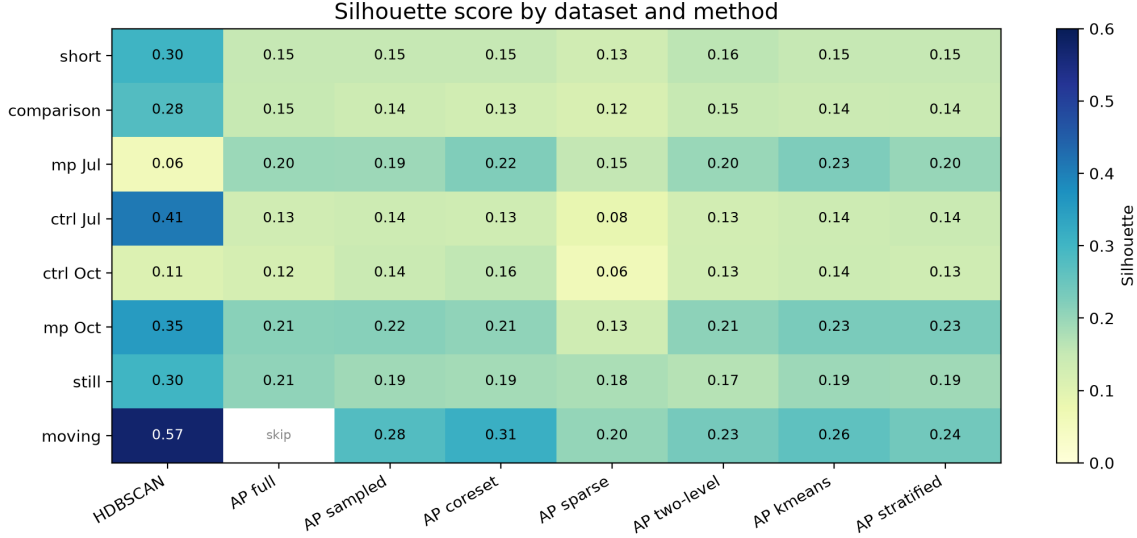


Figure 8: Silhouette score by dataset and method. Higher values indicate stronger geometric separation in the CDF-L1 feature space, but silhouette alone does not imply biological validity.

Table 8: ARI against full AP for datasets where full AP was run. AP full is omitted because it is the reference method.

Dataset	HDBSCAN	AP Sampled	AP Coreset	AP Sparse	AP Two-level	AP K-Means	AP Stratified
short_comparison_test	0.001	1.000	1.000	0.395	0.548	1.000	1.000
comparison_test	0.004	0.472	0.403	0.350	0.347	0.461	0.365
mp_mouse_1_jul	0.087	0.438	0.432	0.378	0.439	0.474	0.455
control_mouse_1_jul	0.000	0.325	0.346	0.288	0.340	0.407	0.384
control_mouse_1_oct	0.006	0.387	0.330	0.272	0.331	0.343	0.392
mp_mouse_1_oct	0.225	0.446	0.457	0.364	0.384	0.466	0.486
still_test	0.267	0.450	0.349	0.377	0.430	0.453	0.402

12.3 Runtime

Table 9 reports wall-clock runtime on a single CPU core, measured directly for every method on every dataset in one version-pinned batch run (no timings are estimated or extrapolated). AP Full is the slowest method by a wide margin – 150–220 s on the six datasets between 13,799 and 23,906 windows – and is not run at all above the 25,000-window threshold. Among scalable methods, HDBSCAN is fastest, followed by AP Two-level and AP Coreset; AP Sampled, AP K-Means, and AP Stratified are moderate (15–50 s); AP Sparse is consistently the slowest scalable method, exceeding AP Full’s own runtime on `mp_mouse_1_oct` (169 s) due to its large approximate KNN graph construction step.

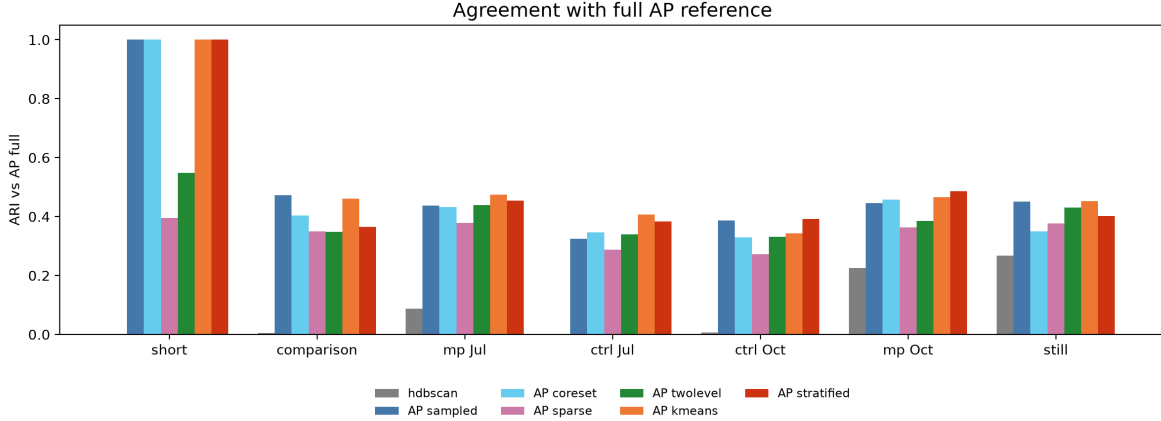


Figure 9: Agreement with full AP measured by ARI. HDBSCAN has low agreement with full AP because it optimizes a density-based objective rather than an exemplar-based AP objective.

Table 9: Wall-clock runtime per dataset and method on a single CPU core. A dash indicates AP Full was skipped ($N > 25,000$).

Dataset	N	HDBSCAN	AP Full	AP Sampled	AP Coreset	AP Two-level	AP Sparse	AP K-Means	AP Stratified
short_comparison_test	1,195	<0.1 s	0.2 s	0.2 s	0.2 s	0.1 s	0.3 s	0.2 s	0.2 s
comparison_test	13,799	1.9 s	55.1 s	17.8 s	22.6 s	16.1 s	14.6 s	32.1 s	27.5 s
mp_mouse_1_jul	19,714	2.4 s	150.9 s	34.5 s	27.2 s	23.6 s	53.7 s	47.6 s	35.2 s
control_mouse_1_jul	23,902	3.4 s	196.3 s	21.5 s	24.4 s	26.4 s	73.0 s	34.3 s	23.2 s
control_mouse_1_oct	23,906	3.3 s	218.8 s	24.4 s	22.8 s	32.4 s	99.1 s	32.2 s	16.3 s
mp_mouse_1_oct	23,866	2.4 s	214.2 s	51.6 s	23.5 s	25.7 s	168.6 s	48.8 s	26.3 s
still_test	23,877	2.2 s	177.2 s	19.5 s	26.3 s	24.2 s	46.4 s	48.6 s	21.0 s
moving_test	47,028	5.7 s	–	46.9 s	31.7 s	65.3 s	153.7 s	31.3 s	33.8 s

12.4 Downstream Behavioral Agreement

ARI alone measures whether two labelings group the same windows together; it does not guarantee that behavioral summaries computed from those labels agree. Because AP full and AP sampled are independent runs, their cluster IDs are not directly comparable, so clusters are first matched via Hungarian assignment on their window-overlap contingency table (maximizing total overlap) before any of the metrics below are computed. For the seven datasets with a full-AP reference, Table 10 reports four such metrics between AP sampled and AP full: the Pearson correlation of per-cluster occupancy (fraction of time in each behavior) and of the flattened first-order transition matrix, both with bootstrap 95% confidence intervals (1,000 resamples over matched cluster pairs); frame-level raster agreement (the fraction of windows where AP sampled’s label, translated through the cluster matching, equals AP full’s label at that same window); and the fraction of AP-full exemplars whose nearest AP-sampled exemplar is the identical window.

Occupancy and transition-matrix correlation both sit around 0.9–1.0 on every dataset except the trivially small `short_comparison_test` case: AP sampled reproduces AP full’s *population-level* behavioral statistics almost exactly, which is the evidence that most directly supports using

Table 10: Downstream agreement between AP sampled and AP full. K full/sampled/matched are the cluster counts from each method and the number of matched pairs found by Hungarian assignment. Occupancy and transition columns show the point estimate with the 95% bootstrap CI in brackets.

Dataset	K (full/sampled/matched)	Occupancy r	Transition r	Raster agree.	Exemplar match
<code>short_comparison_test</code>	9 / 9 / 9	1.000 [1.00, 1.00]	1.000 [1.00, 1.00]	100.0%	100.0%
<code>comparison_test</code>	33 / 28 / 28	0.911 [0.80, 0.97]	0.930 [0.89, 0.96]	59.9%	33.3%
<code>control_mouse_1_jul</code>	42 / 25 / 25	0.764 [0.54, 0.90]	0.931 [0.89, 0.95]	41.5%	9.5%
<code>control_mouse_1_oct</code>	45 / 28 / 28	0.956 [0.92, 0.98]	0.975 [0.96, 0.98]	50.8%	8.9%
<code>mp_mouse_1_jul</code>	40 / 26 / 26	0.927 [0.87, 0.97]	0.965 [0.94, 0.98]	54.6%	10.0%
<code>mp_mouse_1_oct</code>	50 / 30 / 30	0.919 [0.77, 0.97]	0.955 [0.93, 0.97]	50.5%	2.0%
<code>still_test</code>	46 / 27 / 27	0.803 [0.50, 0.92]	0.923 [0.89, 0.95]	53.4%	2.2%

AP sampled as the default scalable method. Frame-level raster agreement and exemplar identity tell a different story – raster agreement sits near 50% and exemplar identity is as low as 2–10% on most datasets – meaning the two methods frequently assign *individual windows* to different specific clusters, or select different representative windows as exemplars, even while the aggregate statistics that matter for behavioral comparison converge. AP sampled should therefore be trusted for population-level summaries (occupancy, transition structure) but not for frame-level or single-instance cluster identity.

Caveat on this table: three of the seven datasets here (`control_mouse_1_jul/oct`, `mp_mouse_1_oct`) are drawn from the MitoPark disease-progression cohort. If this analysis is folded into a manuscript that also makes disease-progression claims, replacing these with open-field-only recordings avoids diluting the biological novelty of a future MitoPark-focused paper.

12.5 Main Observations

Random-sampled AP remains the most reliable general-purpose replacement for full AP: it runs in under a minute on every tested dataset (17–52s across the six non-trivial datasets) and achieves ARI between 0.325 and 0.472 on the seven datasets where full AP is now available (up from three datasets in the previous version of this report; Section 12.4 additionally shows it reproduces full AP’s occupancy and transition-matrix statistics with $r \approx 0.9$ –1.0). No single sampled variant dominates every dataset, though: k-means and stratified sampling each exceed random sampling’s ARI on several datasets—stratified reaches 0.486 versus 0.446 for random on `mp_mouse_1_oct`, and k-means reaches 0.407 versus 0.325 for random on `control_mouse_1_jul`—so they are worth considering when sample composition matters.

Coreset sampling shows a different trade-off: it tends to improve silhouette (notably on `mp_mouse_1_jul` and `moving_test`) but does not always improve ARI. The greedy farthest-point selection covers the feature space more uniformly, which helps geometric separation but does not necessarily match the exemplars that full AP would choose.

Sparse AP consistently over-fragments. With full AP now computed on seven datasets, the cluster-count ratio is consistently 2–3 $\times$ higher than full AP (from 2.1 $\times$ on `short_comparison_test` to 3.2 $\times$ on the two `control_mouse_1` sessions), and this is not easily fixed by tuning—it is a consequence of the graph structure (Section 9). It is also the slowest scalable method on five of the eight datasets. It is retained for completeness but is not recommended for production use.

HDBSCAN occasionally achieves higher silhouette than any AP variant (e.g. `moving_test`: 0.574), but its ARI against full AP is essentially zero on most datasets (including exactly 0.000

on `control_mouse_1_jul`), and it assigns 54–66% of windows as noise on high-activity datasets. It reflects a genuinely different objective and should be treated as a separate exploratory tool rather than a drop-in replacement.

Figure 5. Random-sampled AP is a reliable scalable replacement for full AP

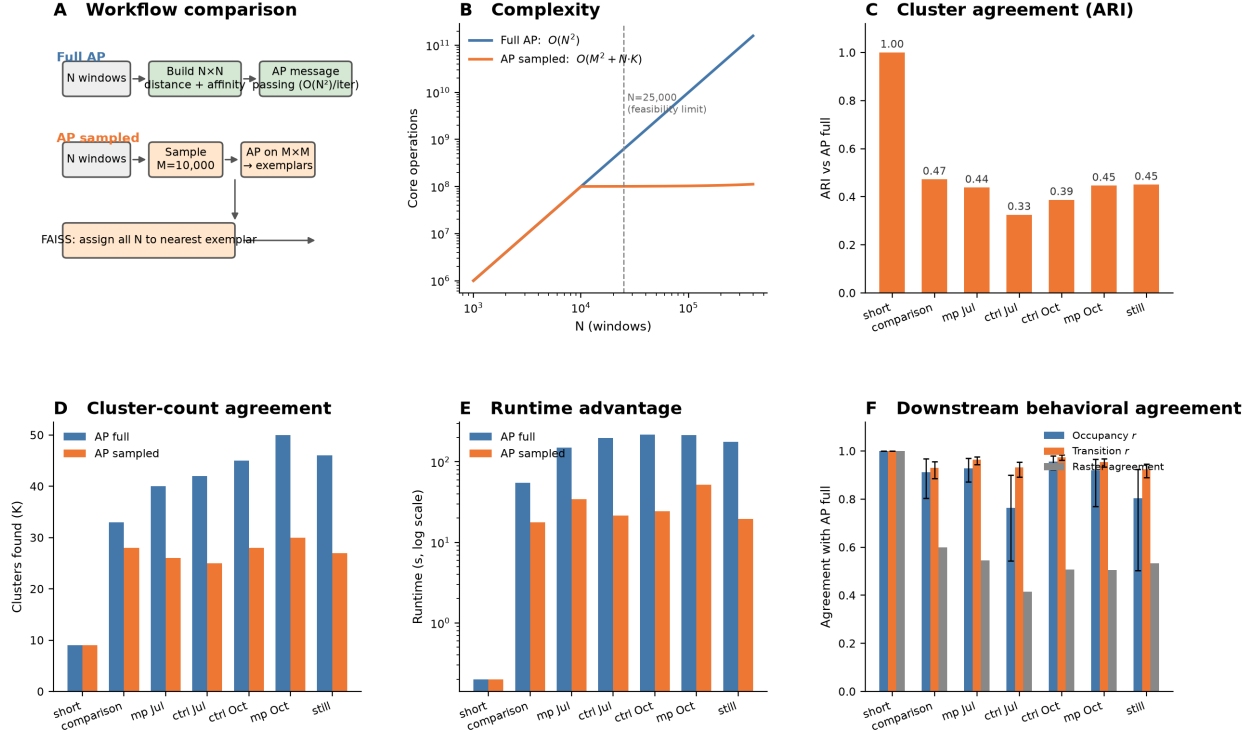


Figure 10: Random-sampled AP as a scalable replacement for full AP, summarized across the results of Sections 12–12.4. (A) Workflow: full AP builds a dense $N \times N$ affinity matrix and runs message passing over all N windows; AP sampled runs the same message passing on a fixed-size sample of $M = 10,000$ windows and assigns every window to its nearest exemplar via FAISS. (B) The resulting complexity gap: full AP is $O(N^2)$; AP sampled is $O(M^2 + N \cdot K)$, effectively flat past $N \approx 10,000$. (C) ARI between AP sampled and AP full (Table 8). (D) Cluster counts found by each method (Tables 6, 8). (E) Wall-clock runtime, log scale (Table 9). (F) Downstream behavioral agreement (Table 10): occupancy and transition-matrix correlation (error bars are the 95% bootstrap CI) stay near 0.9–1.0, while frame-level raster agreement is substantially lower, showing that AP sampled preserves population-level statistics without reproducing full AP’s frame-by-frame labeling.

13 Discussion

13.1 Recommended Current Method

Random-sampled AP is the current recommended default for replacing the MATLAB pipeline. It runs in 17–52s on all tested datasets, stays close to full AP where comparison is possible, and reproduces full AP’s population-level occupancy and transition statistics with $r \approx 0.9$ –1.0 (Section 12.4). For applications where rare behavioral motifs—tremor, impaired gait—are scientifically important, stratified or coreset sampling is preferable: both ensure that small behavioral clusters are represented in the AP sample regardless of how infrequently they occur. K-means sampling is a reasonable middle ground that selects cluster-central rather than boundary points and showed the best ARI on `mp_mouse_1_jul` and `control_mouse_1_jul`.

Full AP remains the ground-truth reference and should be run whenever N is small enough (under $\sim 25,000$ windows on a machine with 18+ GB RAM; this threshold was raised from 20,000/12 GB after moving to a 48 GB machine, and is set conservatively below the point where full AP’s four simultaneous $N \times N$ matrices would exceed available memory).

13.2 Limitations

The histogram representation throws away temporal structure within each 300 ms window. Two windows with the same amplitude distribution but different within-window dynamics look identical to the pipeline—this is likely fine for coarse locomotor categories but could miss rhythmic features like tremor. The fixed MATLAB-derived bin edges are another constraint: they were calibrated for specific arena and sensor configurations, and may not be optimal for MitoPark mice where disease alters the distribution of accelerations.

A less obvious limitation is that silhouette score rewards geometric compactness, which is not the same as biological meaningfulness. HDBSCAN’s high silhouette on some datasets reflects the fact that it discards ambiguous windows as noise, not that its clusters are more biologically valid. Downstream validation—e.g. comparing cluster usage between control and MitoPark mice across weeks—is the real test that silhouette cannot replace; Section 12.4 is a first step in that direction, though it validates AP sampled against AP full rather than against ground-truth behavioral labels.

A reproducibility limitation surfaced directly while preparing this version of the report: re-running AP full on `mp_mouse_1_jul` in a freshly installed environment (rather than the environment that originally produced the cached results) changed the cluster count from 41 to 40, even though AP full has no randomness beyond a fixed `random_state`. This is almost certainly scikit-learn version drift, since the previous `requirements.txt` did not pin exact package versions. All results in this version of the report come from a single run in an environment with pinned versions (numpy 2.5.2, scipy 1.18.1, pandas 3.0.5, scikit-learn 1.9.0, hdbscan 0.8.44, faiss-cpu 1.15.0), and `requirements.txt` now records these versions so future re-runs are exactly reproducible. A related issue was found and fixed in `batch_compare.py`: AP sampled’s preference is only aligned to AP full’s exact preference when AP full is freshly computed in the same batch-compare session, so cached or skipped AP-full runs silently left AP sampled using an unaligned, independently estimated preference; this affected the AP-sampled results for four datasets (up to 97% of window labels differed once corrected) and has been fixed by re-running AP full and AP sampled together for every dataset reported here.

14 Conclusion

The pipeline matches the MATLAB AP output on small datasets ($N \leq 25,000$, raised from 20,000 in the previous version of this report) and extends cleanly to larger recordings through subsampling. Of the scalable variants, random-sampled AP is the most battle-tested and is now validated not only by ARI but by direct agreement on occupancy and transition statistics (Section 12.4); k-means and stratified sampling remain competitive alternatives, now with full timing data recorded across all eight datasets. The next priorities are: (1) re-verifying MATLAB-vs-Python equivalence under the newly pinned dependency versions, since the environment used to originally generate the MATLAB comparison predates the reproducibility fixes described in Section 13.2; (2) resolving whether the MitoPark-derived datasets used in Section 12.4 should be replaced with open-field-only recordings if this analysis is folded into a manuscript that also makes disease-progression claims; and (3) an end-to-end run on a long-duration (≥ 20 hour) recording to demonstrate scalability beyond the current largest dataset (`moving_test`, 47,028 windows).

Implementation Notes

This document is based on the current local contents of:

- `Python_Pipeline/scripts/prepare_test_data.py`
- `Python_Pipeline/scripts/clustering_pipeline.py`
- `Python_Pipeline/scripts/batch_compare.py`
- `Python_Pipeline/scripts/compare_results.py`
- `Python_Pipeline/scripts/recompute_ap_exemplars.py` – recovers AP exemplar indices (discarded by `batch_compare.py`) and produces a self-consistent (AP full, AP sampled) pair per dataset for Section 12.4
- `Python_Pipeline/scripts/fig5f_downstream_agreement.py` – computes the cluster matching, occupancy/transition correlation, raster agreement, and exemplar-overlap metrics in Table 10
- `Python_Pipeline/requirements.txt` – now pins exact package versions (see Section 13.2)
- `Python_Pipeline/results/method_comparison.csv`
- `Python_Pipeline/results/method_comparison_report.txt`
- `Python_Pipeline/results/fig5f_downstream_agreement.csv`

References

- [1] Frey, B. J. and Dueck, D. (2007). Clustering by passing messages between data points. *Science*, 315(5814):972–976.
- [2] Johnson, J., Douze, M., and Jégou, H. (2021). Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 7(3):535–547.

- [3] Campello, R. J. G. B., Moulavi, D., and Sander, J. (2013). Density-based clustering based on hierarchical density estimates. In *Pacific-Asia Conference on Knowledge Discovery and Data Mining*, pages 160–172.
- [4] Rubner, Y., Tomasi, C., and Guibas, L. J. (2000). The earth mover’s distance as a metric for image retrieval. *International Journal of Computer Vision*, 40(2):99–121.
- [5] Rousseeuw, P. J. (1987). Silhouettes: A graphical aid to the interpretation and validation of cluster analysis. *Journal of Computational and Applied Mathematics*, 20:53–65.
- [6] Hubert, L. and Arabie, P. (1985). Comparing partitions. *Journal of Classification*, 2(1):193–218.
- [7] Pedregosa, F. et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830.